Student Name: Sarraj Alsammak
Student ID: 2640297

AC51047 - Advanced Big Data Analysis (SEM 1 24/25)
Assessment (1)
Big Data Storage and Data warehousing.

Introduction

For this assignment, we will explore the use of NoSQL databases, data warehousing, and cloud storage solutions to efficiently handle and analyze a large dataset. The dataset provided, stored in store.csv, contains transactional data from a store that sells various stationery and hardware products, capturing a range of details like product specifications, customer demographics, store locations, sales, and more. Leveraging Amazon Web Services (AWS) tools and SQL-based querying methods, we aim to design solutions that optimize storage, retrieval, and analytical processes for this data.

Abstract

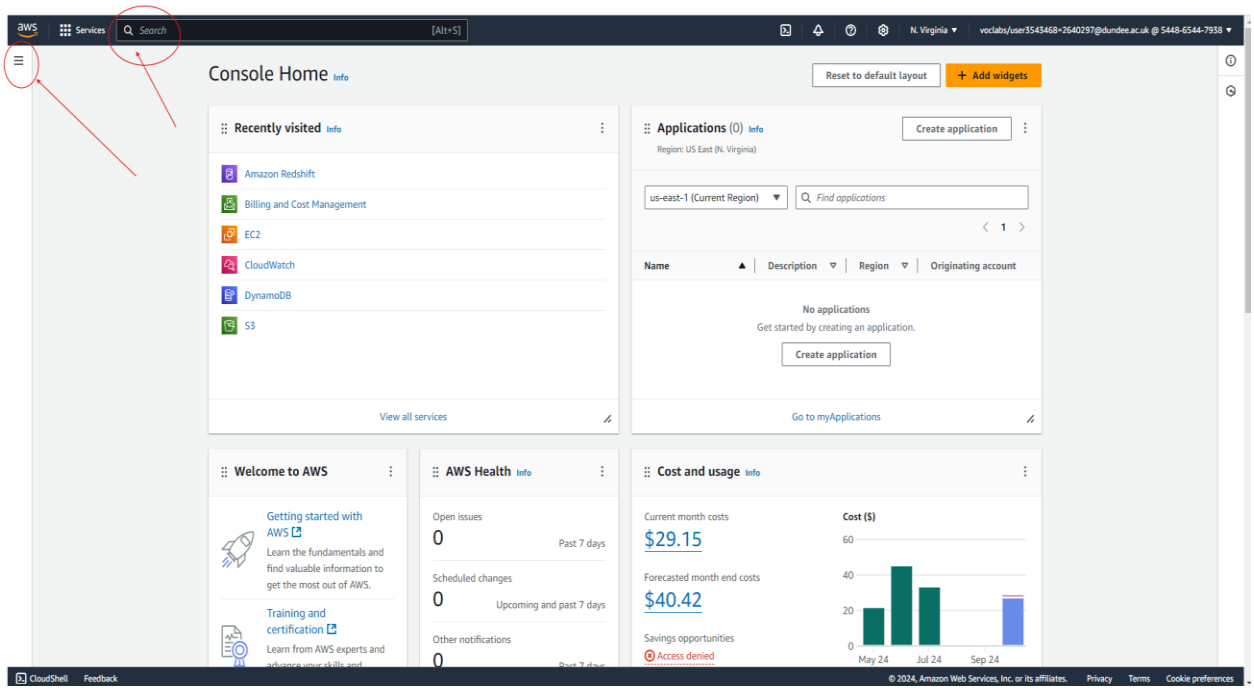
The assignment is divided into three primary tasks:

1. **Cloud Storage with Amazon S3**: We will begin by setting up a cloud storage solution using Amazon S3 to securely store the dataset.
2. **NoSQL Database Implementation with DynamoDB or Cassandra**: Using a NoSQL database (DynamoDB), we will model the data to accommodate large-scale storage and efficient access patterns. Here, we will carefully select partition and sort keys to optimize data retrieval for specific queries, balancing the storage design with query performance. Additionally, we will identify queries that can and cannot be efficiently executed on our table design, proposing secondary indexes or alternative designs to enhance query efficiency.
3. **Data Warehousing and Analysis with Amazon Redshift or Apache Hive**: Finally, we will build an external data warehouse table as a wrapper for the NoSQL database, linking it to Amazon Redshift or Apache Hive for complex data analysis. Through SQL queries on this table, we will derive meaningful insights from the data, such as identifying top-performing product categories, analyzing regional sales, and exploring customer purchasing patterns. To further streamline our analyses, we will create data cubes that simplify multi-dimensional analysis, providing faster access to key metrics and aggregations.

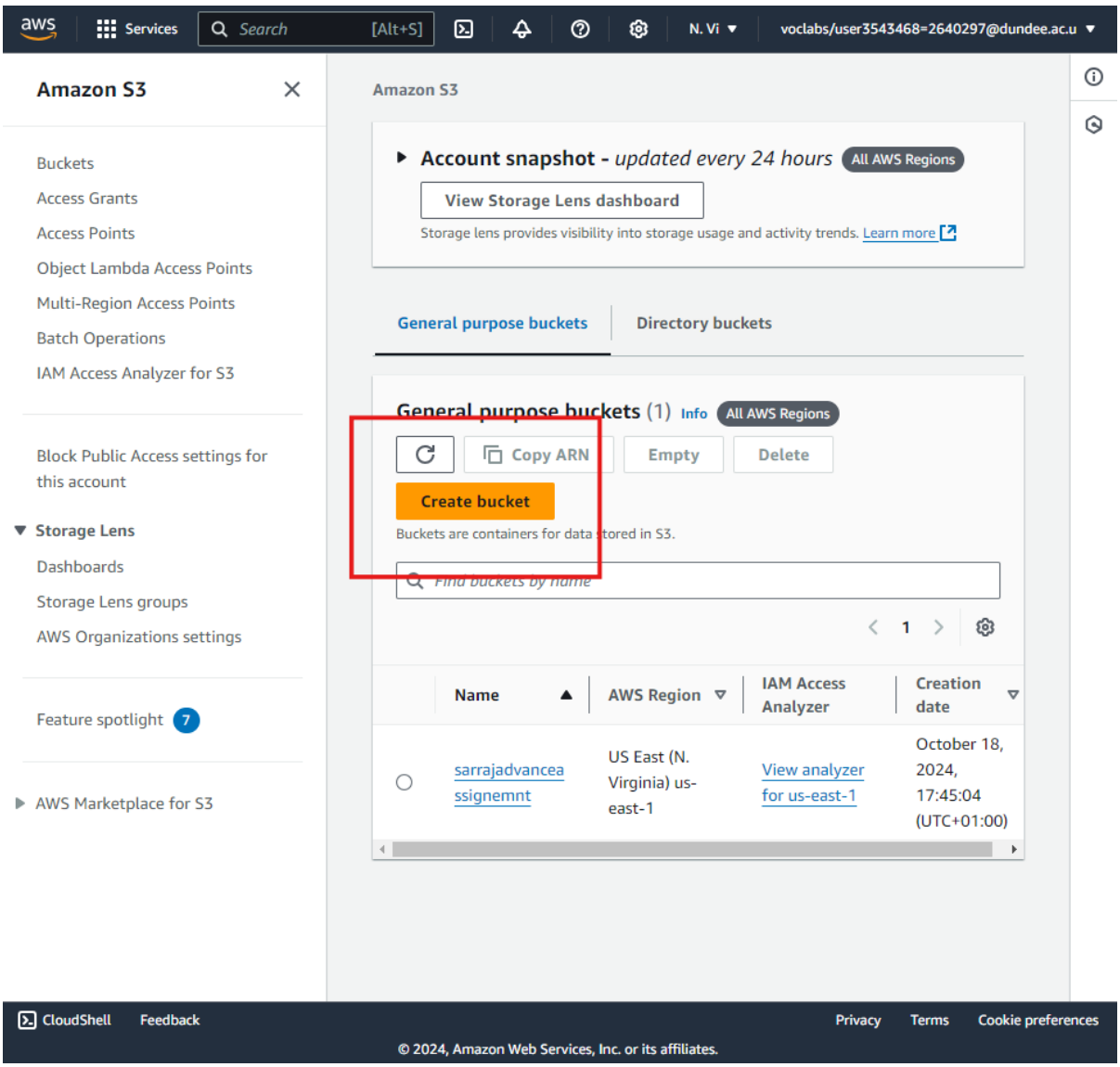
Part one: Creating a S3 Bucket and uploading the store.csv file into the bucket

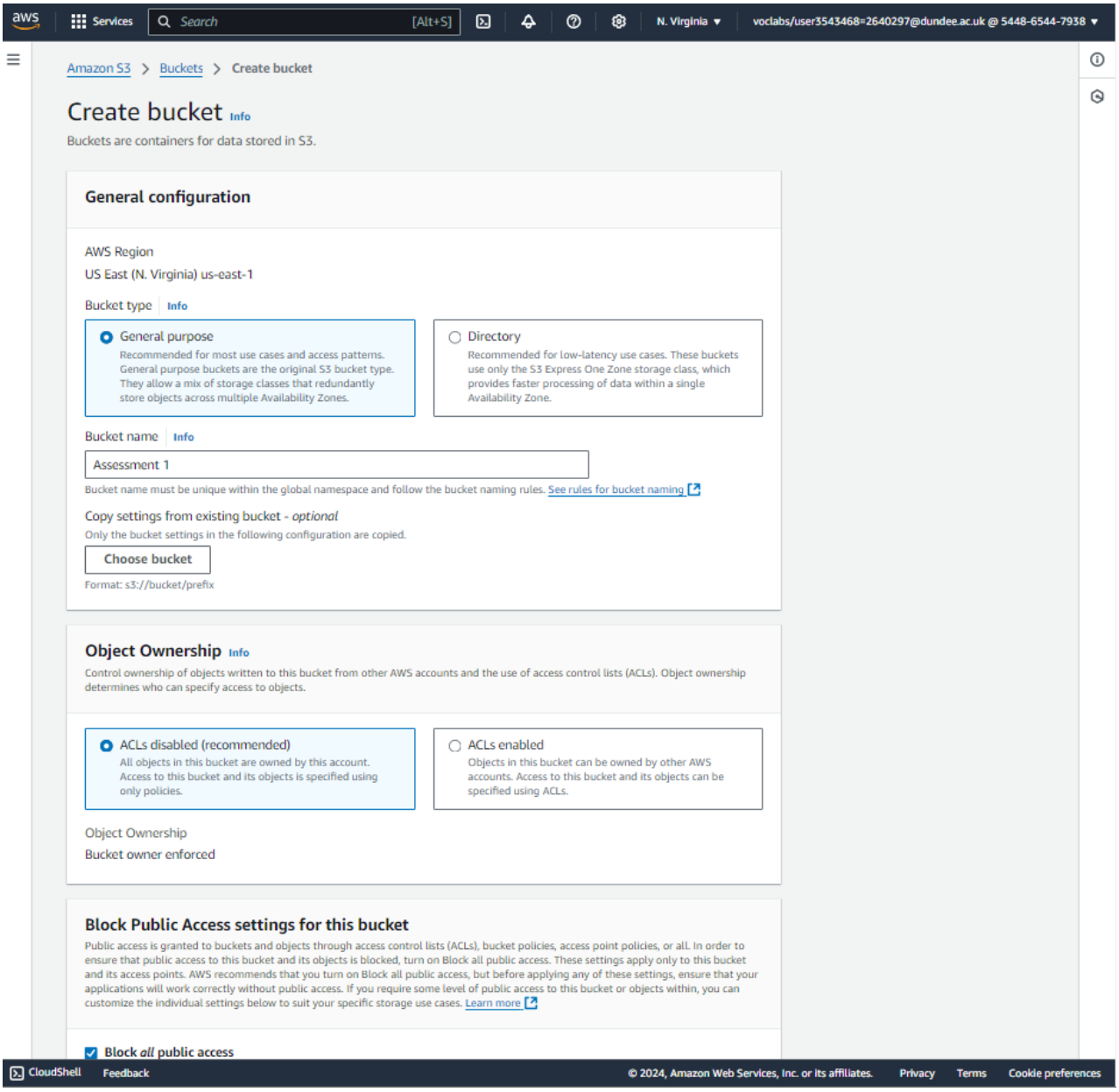
→ **Create a Bucket.**

In the AWS console Home, we search for S3 service, or we can search for S3 service.



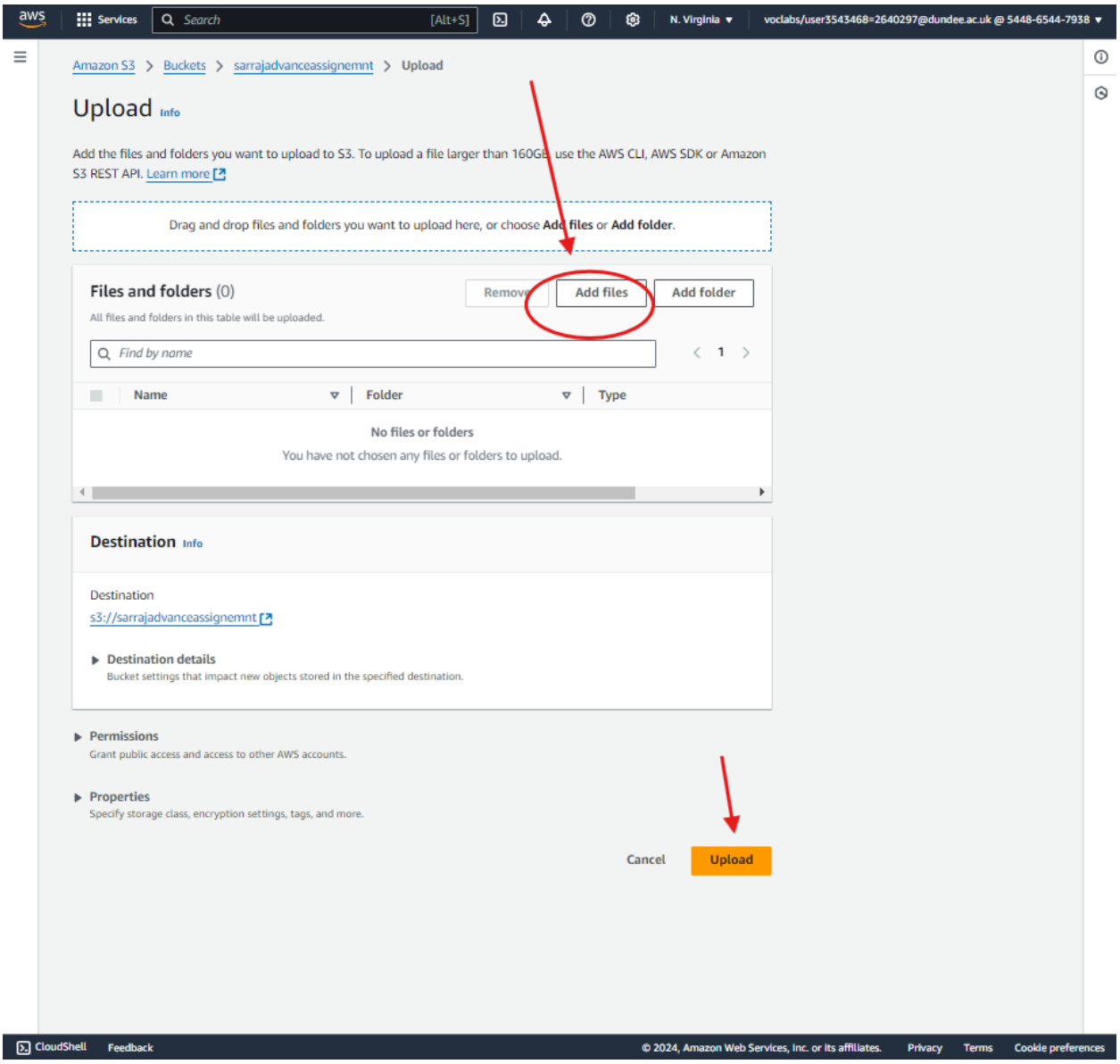
Then we choose it and navigate it

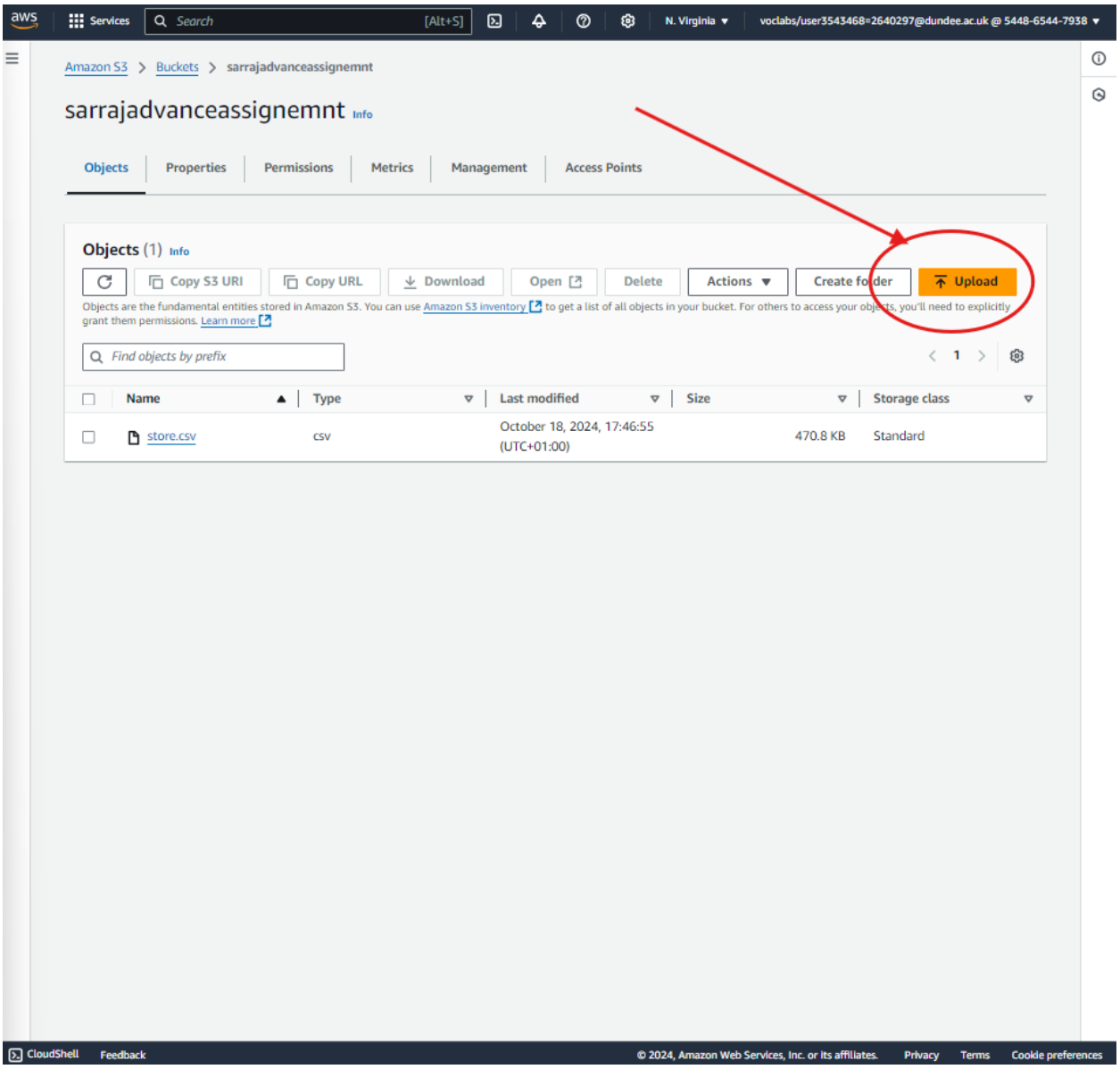




→ **Uploading file.**

After Creating Our S3 bucket we open it and upload our file.


aws
Services
Search
[Alt+S]
N. Virginia
voclabs/user3543468=2640297@dundee.ac.uk @ 5448-6544-7938
Amazon S3 > Buckets > sarrajadvanceassignemnt > Upload
Upload Info
Add the files and folders you want to upload to S3. To upload a file larger than 160GB, use the AWS CLI, AWS SDK or Amazon S3 REST API. Learn more
Drag and drop files and folders you want to upload here, or choose Add files or Add folder.
Files and folders (0)
All files and folders in this table will be uploaded.
Remove
Add files
Add folder
Find by name
1
Name
Folder
Type
No files or folders
You have not chosen any files or folders to upload.
Destination Info
Destination
s3://sarrajadvanceassignemnt
Destination details
Bucket settings that impact new objects stored in the specified destination.
Permissions
Grant public access and access to other AWS accounts.
Properties
Specify storage class, encryption settings, tags, and more.
Cancel
Upload
CloudShell
Feedback
© 2024, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

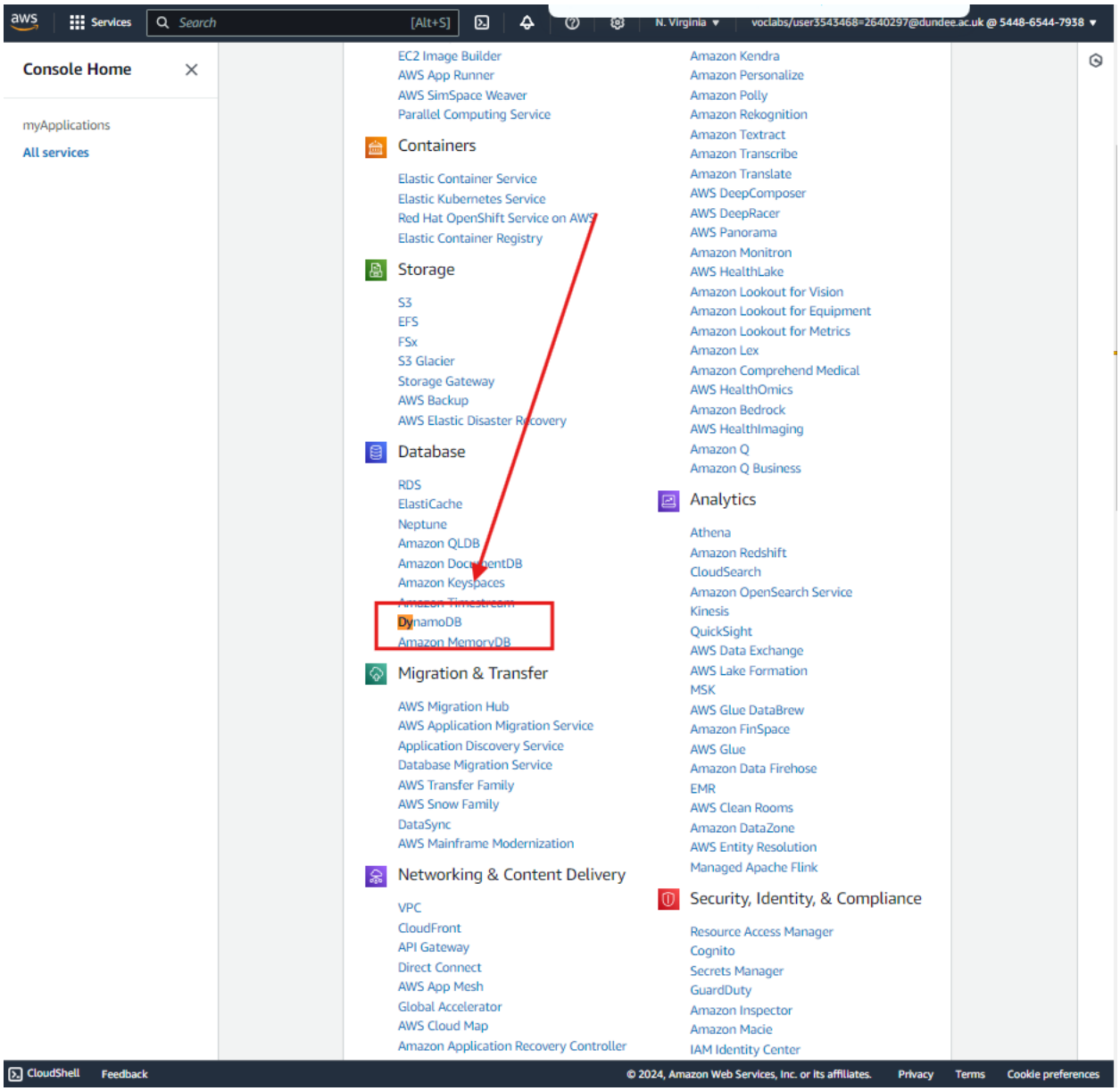


After finishing that store.CVS is uploaded to the S3 Bucket.
After reviewing My table i have added new column to be a Partition key (Order Id_Profit).

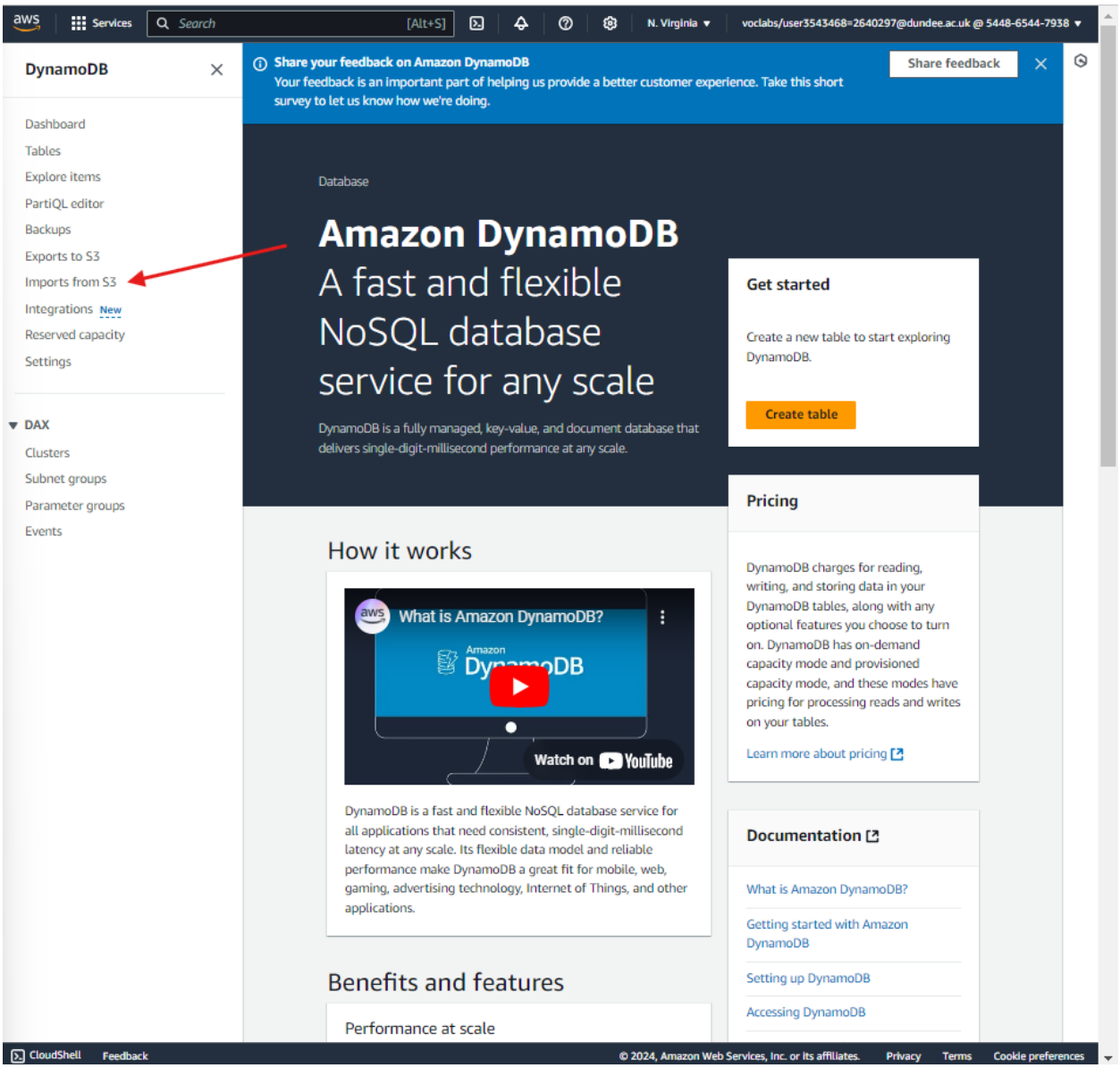
Part Two: NoSQL Database Implementation with DynamoDB

→ **Create/Importing a Table.**

In the AWS console Home, we search for DynamoDB Database, or we can search for DynamoDB Database.



After opening DynamoDB we click on import from S3



After that we open S3 to get **Source URL,** then we import the table

Dashboard
Tables
Explore items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations New
Reserved capacity
Settings

DAX
Clusters
Subnet groups
Parameter groups
Events

DynamoDB > Imports from S3 > Import from S3

Step 1
Import options

Step 2
Specify table details

Step 3 - optional
Configure table settings

Step 4
Review and import

Import options Info

Import data from an S3 bucket to a new DynamoDB table. The table can be in any AWS Region and owned by any account with read permissions. Extra charges may apply. Learn more

Import details Info

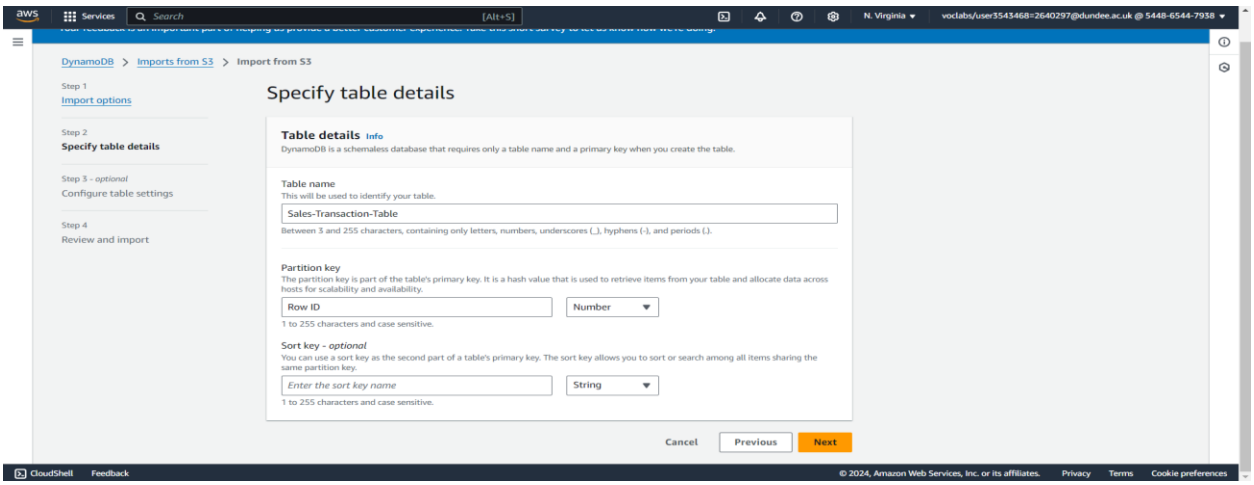
Source S3 URL
Choose the Amazon S3 storage source to import from. You can specify a bucket or a path within the bucket.
Enter S3 source URL | View | Browse S3
Example formats: s3://countries or s3://countries/territories/

S3 bucket owner
- This AWS account (544865447938)
- A different AWS account

Import file compression
Choose the compression type that matches your source S3 data.
- No compression
- GZIP
- ZSTD

Import file format Info
Choose the file format that matches your Amazon S3 storage source.
- DynamoDB JSON
- Amazon Ion
- CSV

Cancel | Next



1. What did you put as a partition key in this table and why? What (if anything) did you put as a sort key and why? What other attributes could have you put for partition and sort key, compared to your solution?

My keys are:

1. Partition key is The **Sales-Transaction-Table01**, why? Because it's a unique identifier on the table and it's comping between the most required feature.
2. Sort Key **IT IS NOT REQUIRED** due to the fact that the Partition key is unique.

2. Give at least two examples of queries that could be efficiently executed on your table. Describe the queries both using English language and the PartiQL language. Also describe how you would perform these queries on the table that you have created (attach screenshots if you have used the DynamoDB Dashboard interface or attach the commands if you have used the AWS CLI interface from the AWS terminal).

> **Hint:** To know whether a query can be 'efficiently executed' or not, you can try running your query in PartiQL editor on AWS DynamoDB service and observe whether you get all the results or just first 200 results, and in what time.

a. SELECT * FROM "Sales-Transaction-Table01"
WHERE Profit> '89' and Region = 'West'

Completed
Started on 10/28/2024, 11:52:39 AM
Elapsed time 143ms

b. SELECT * FROM " Sales-Transaction-Table01"
WHERE City = 'Redlands'

Started on 10/28/2024, 12:03:36 PM
Elapsed time 149ms

3. Give at least two examples of queries that could not be efficiently executed on your table. Describe the queries both using the English language and the PartiQL language. Explain also why the queries in this part could not be efficiently executed.

> **Hint:** Again, to judge whether a query can be 'efficiently executed' or not, try running a query and look at the results (and the information about the result) in the PartiQL editor.

a. SELECT * FROM " Sales-Transaction-Table01" WHERE "Country" = 'United States' and begins_with("Product Name", 'SANFORD')

Completed
Started on 10/28/2024, 12:15:00 PM
Elapsed time 384ms

b. SELECT "Product Name", "Region", "Profit" , "County"
FROM " Sales-Transaction-Table01" where Country ='United States'

Completed
Started on 10/28/2024, 12:19:24 PM
Elapsed time 273ms

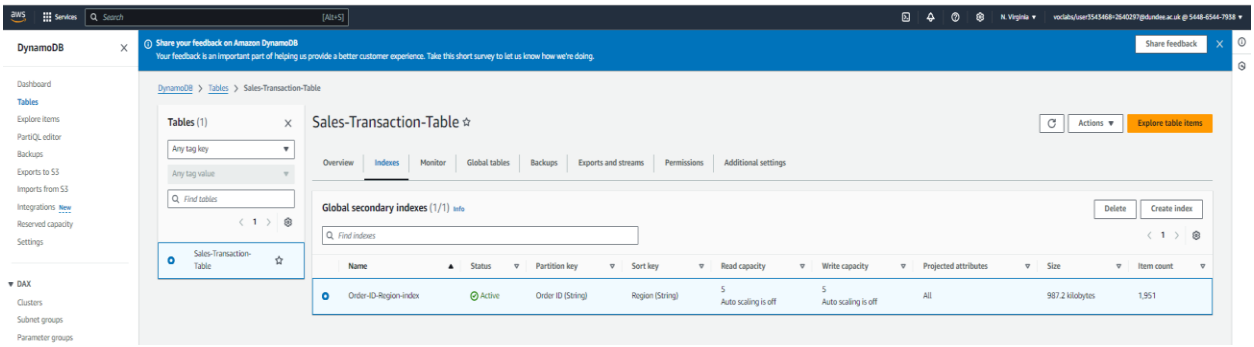
As well we can use aggregation functions like **SUM or GROUP BY** because these operations cannot be operated in DynamoDB.

Example

```
SELECT "Product Category", SUM("Sales") AS "Total Sales" FROM " Sales-Transaction-Table01" GROUP BY "Product Category";
```

4. What would you do to make the execution of the queries for the part c more efficient? How would this make the execution of the queries faster? Attach screenshots if you have used the DynamoDB Dashboard, or the actual commands performed if you have used the AWS CLI interface from the AWS terminal.

   By using Secondary Indexes, we can use Sort key, Global Secondary Index (GSI).

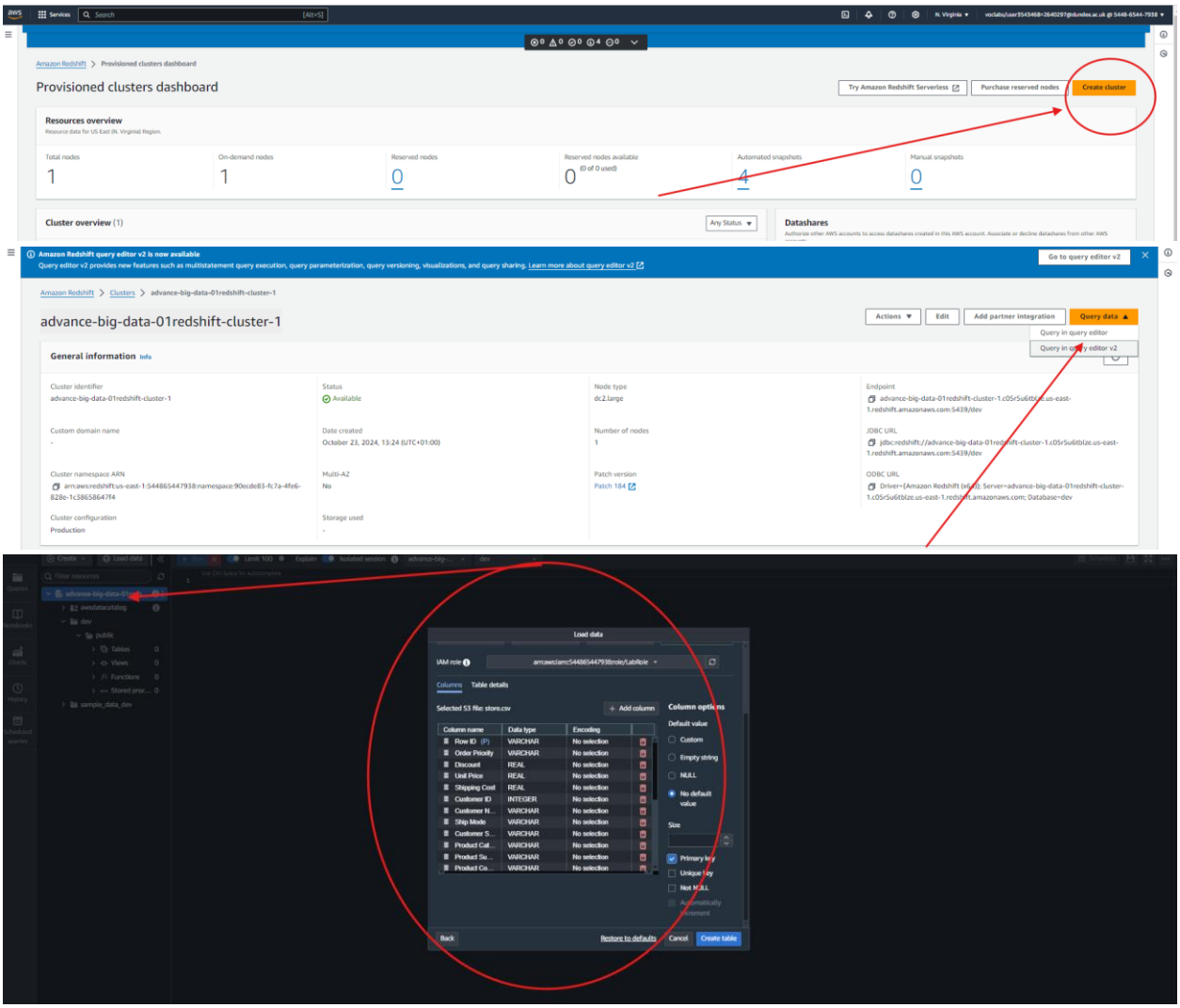


Part Three: Data Warehousing and Analysis with Amazon Redshift

For using the Query editor, we need before to prepare the Redshift, cluster and to connect data with databases.

So, we will be using Data warehouse OLAP:

1. The dev Database will be used due to the lack of permissions for creating Database.
2. Load the data (table) from S3 and revise the data.




```
COPY dev.public."Sales-Transaction" FROM 's3://sarrajadvanceassignemnt/store.csv'
IAM_ROLE 'arn:aws:iam::544865447938:role/LabRole' FORMAT AS CSV DELIMITER ','
QUOTE '"' IGNOREHEADER 1 DATEFORMAT 'DD/MM/YYYY' REGION AS 'us-east-1'
```

a. Write SQL queries on the table to answer the following questions:

1. How many different product categories do there exist in the dataset?

We do have 3 different categories which is:

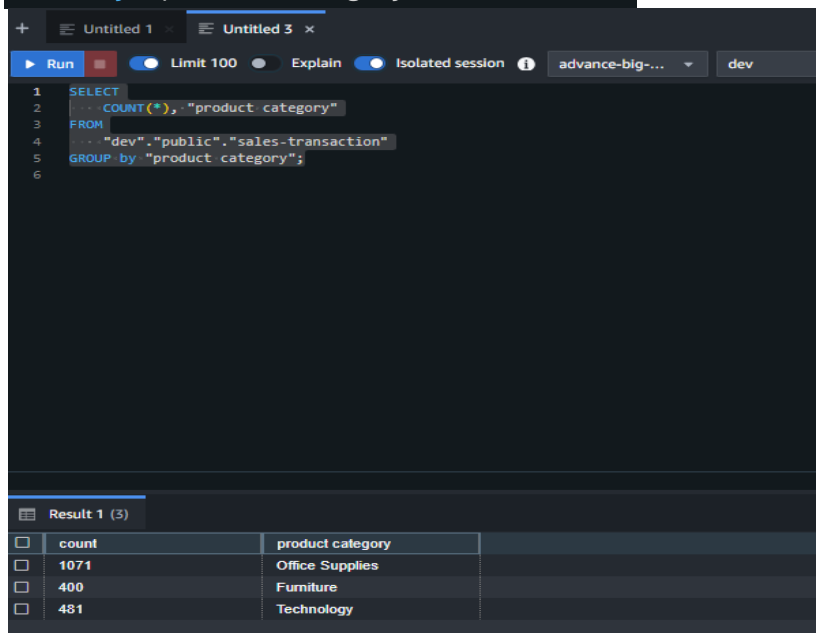
A. Office supplies 1071.

B. Furniture 400.

C. Technology 481.

We can get this result for this Query:

```
SELECT
    COUNT(*), "product category"
FROM
    "dev"."public"."sales-transaction"
GROUP by "product category";
```



But to know the numbers of Categories

```
SELECT
    COUNT(*)
FROM (
    SELECT
        COUNT(*), "product category"
    FROM
        "dev"."public"."sales-
transaction"
    GROUP BY
        "product category"
) AS category_groups;
```

2. How many orders were there in each of the regions?
   In Each region we have orders as follows:

order_count	region
336	West
323	East
400	Central
326	South

We can get this result for this Query:

```
SELECT
    COUNT(DISTINCT "order id") AS order_count,
    "region"
FROM
    "dev"."public"."sales-transaction"
GROUP BY
    "region";
```

```sql
SELECT
    COUNT(DISTINCT "order id") AS order_count,
    "region"
FROM
    "dev"."public"."sales-transaction"
GROUP BY
    "region";
```

Result 1 (4)

order_count	region
336	West
323	East
400	Central
326	South

3. Which product category has the highest total sales amount?

Based on the product category we have Technology the highest one

```sql
SELECT
    SUM(sales) as MAX_Sales, "product category"
FROM
    "dev"."public"."sales-transaction"
GROUP by "product category"
ORDER by MAX_Sales DESC
LIMIT 1;
```

```sql
SELECT

    sum(sales) as MAX_Sales, "product category"

FROM

    "dev"."public"."sales-transaction"

GROUP by "product category"
ORDER by MAX_Sales DESC
limit 1
```

Result 1 (1)

max_sales	product category
712264.9495639801	Technology

4. Which states have recorded the biggest and the smallest profit?
   - → California 37421.9
   - → Noth Carolina –19427.9

```sql
SELECT * from (SELECT SUM(profit) as profit,"state or province" from "dev"."public"."sales-transaction" group by "state or province"

ORDER BY profit DESC LIMIT 1)

UNION ALL

SELECT * from (SELECT SUM(profit) as profit,"state or province" from "dev"."public"."sales-transaction" group by "state or province"

ORDER BY profit ASC LIMIT 1)
```

Untitled 1 | Untitled 3

Run | Limit 100 | Explain | Isolated session | advance-big-... | dev

```
SELECT * from (SELECT SUM(profit) as profit,"state or province" from "dev"."public"."sales-transaction" group by "state or province"

ORDER BY profit DESC LIMIT 1)

UNION ALL

SELECT * from (SELECT SUM(profit) as profit,"state or province" from "dev"."public"."sales-transaction" group by "state or province"

ORDER BY profit ASC LIMIT 1)
```

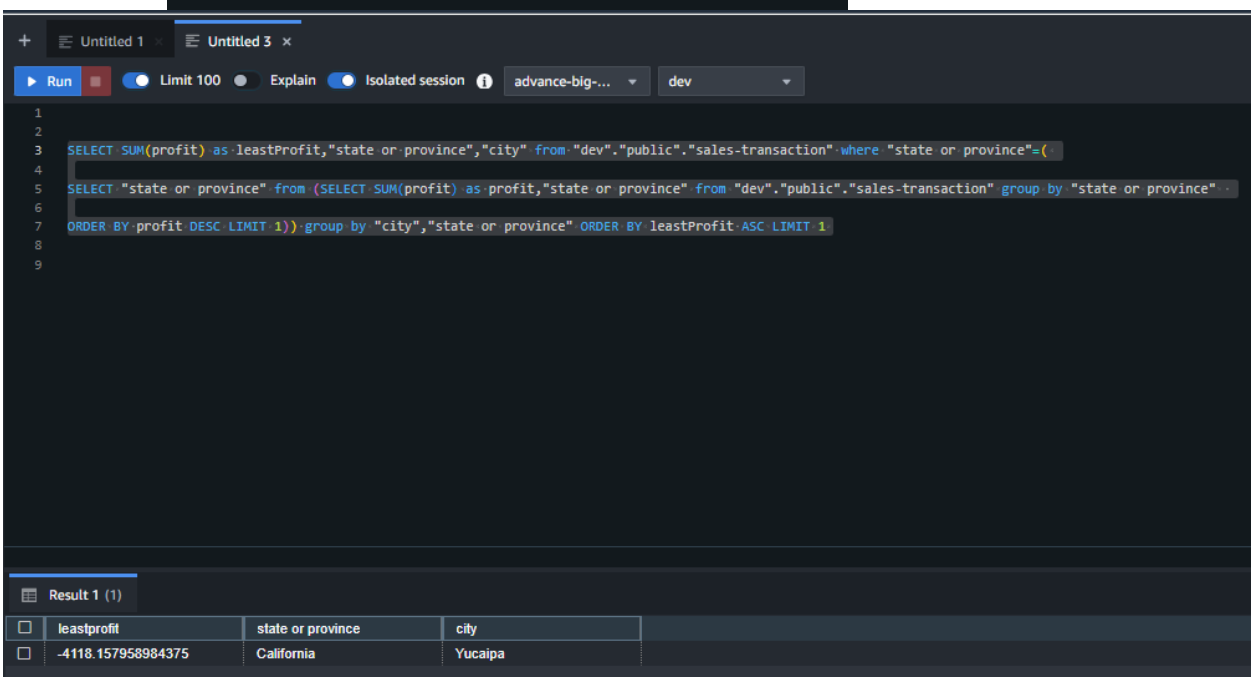
Result 1 (2)

profit	state or province
37421.96042372286	California
-19427.914732933044	North Carolina

5. Which city in the most profitable state has the least profit?

→ California, Yucaipa = -4118.15

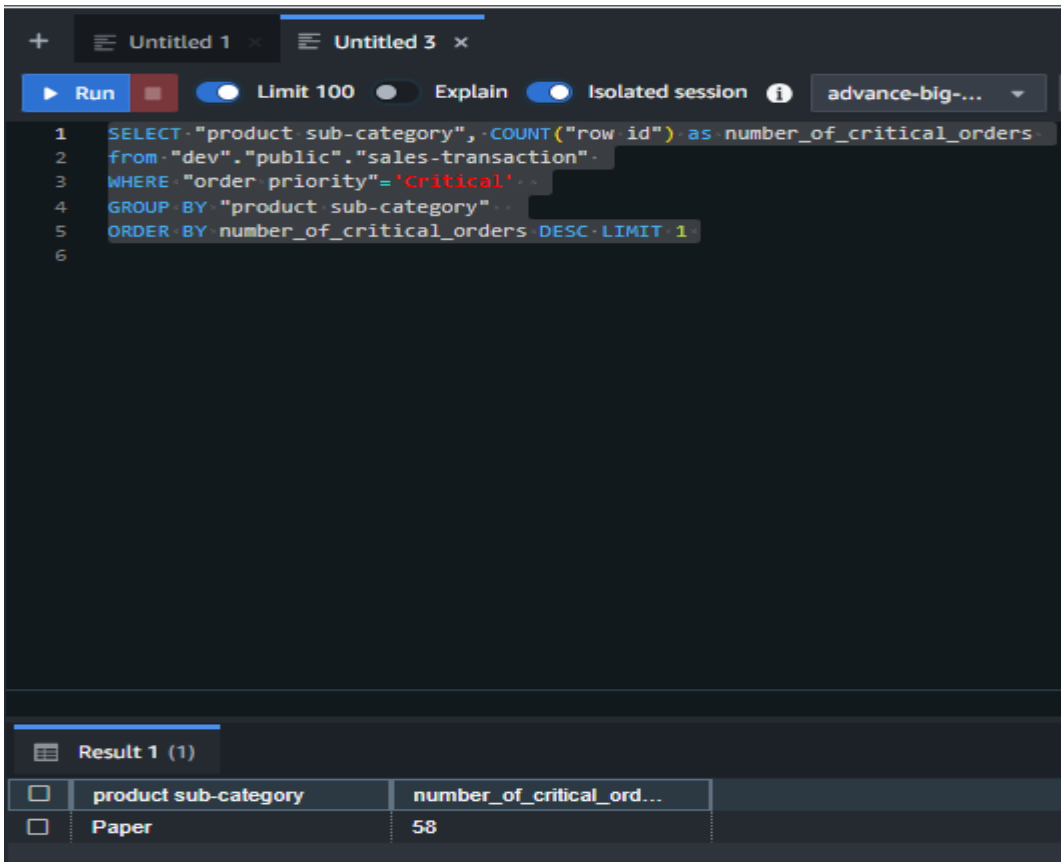
```
SELECT SUM(profit) as leastProfit,"state or
province","city" from
"dev"."public"."sales-transaction" where
"state or province"=(

SELECT "state or province" from (SELECT
SUM(profit) as profit,"state or province"
from "dev"."public"."sales-transaction"
group by "state or province"

ORDER BY profit DESC LIMIT 1)) group by
"city","state or province" ORDER BY
leastProfit ASC LIMIT 1
```



6. Which product subcategory had the most orders that were of the critical priority?

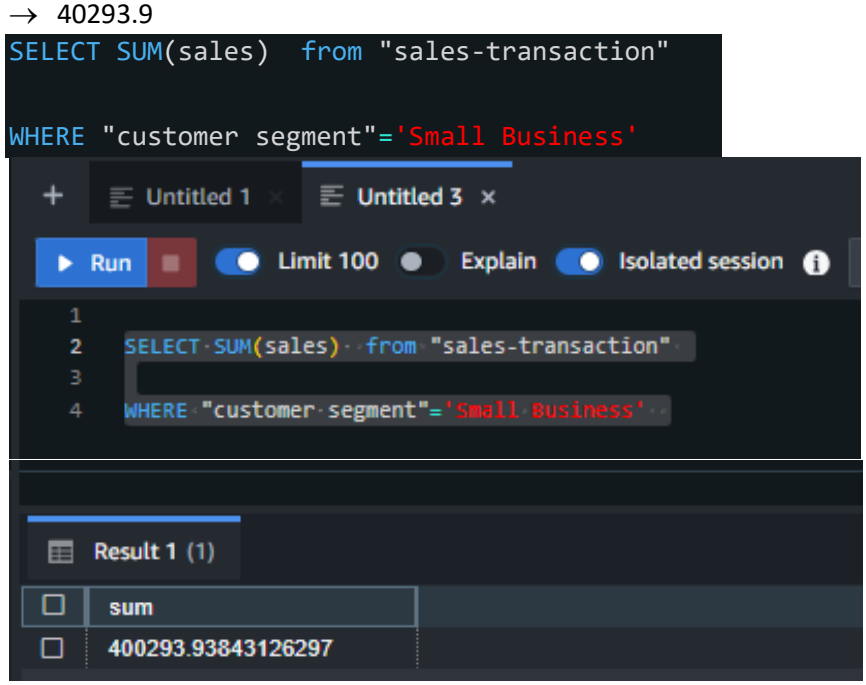
→ Paper 58.

```
SELECT "product sub-category", COUNT("row id") as
number_of_critical_orders
from "dev"."public"."sales-transaction"
WHERE "order priority"='Critical'
GROUP BY "product sub-category"
ORDER BY number_of_critical_orders DESC LIMIT 1
```



7. What is the total amount of sales for 'Small Business' customer segment?

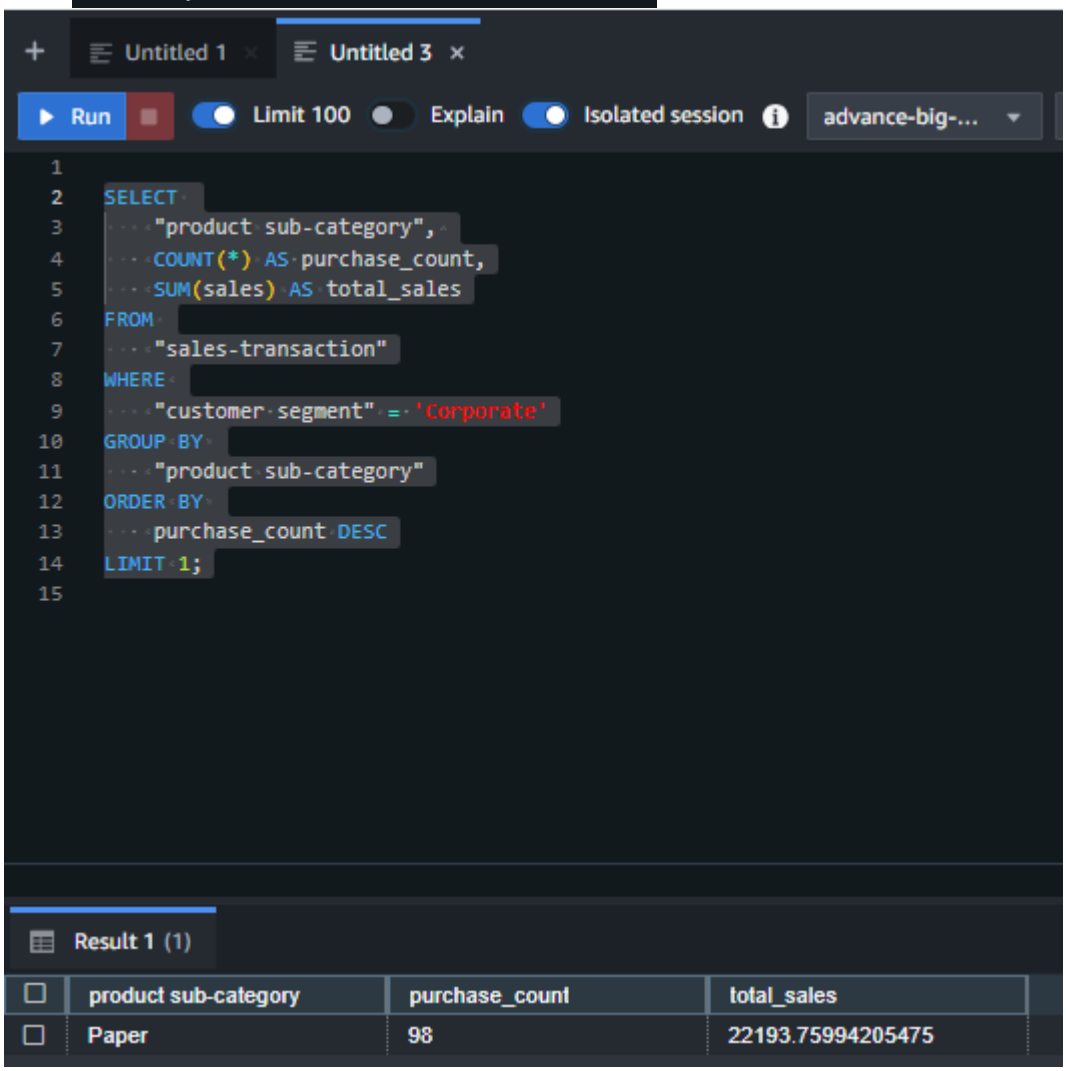
   → 40293.9



8. What product sub-category was the most often purchased in the 'Corporate' customer segment, and what is the total amount of sales in all these transactions?

   → It was paper, 98 action and total sales of 22193.7

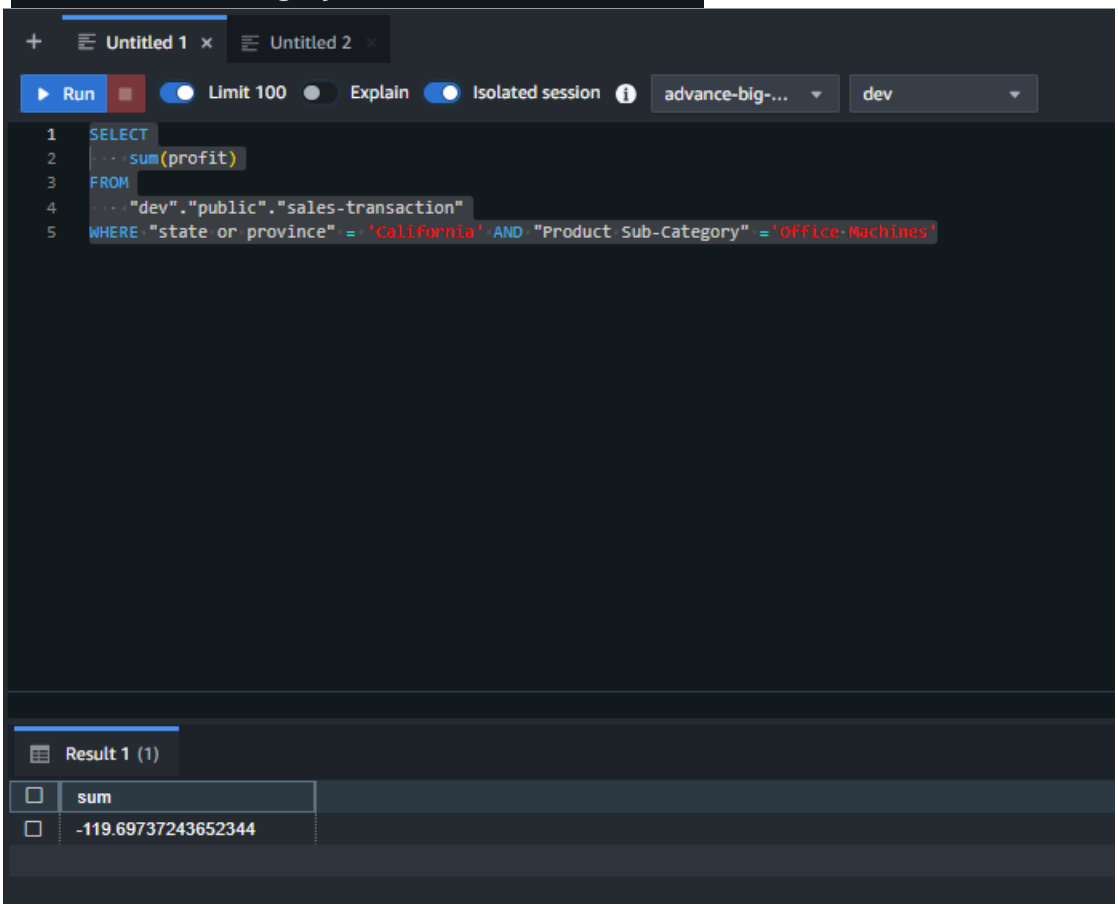
```sql
SELECT
    "product sub-category",
    COUNT(*) AS purchase_count,
    SUM(sales) AS total_sales
FROM
    "sales-transaction"
WHERE
    "customer segment" = 'Corporate'
GROUP BY
    "product sub-category"
ORDER BY
    purchase_count DESC
LIMIT 1;
```



9. What is the total profit for all the products of the 'Office Machines' subcategory in the 'California' state?

→ The total profit is – 119.69.

```
SELECT
    sum(profit)
FROM
    "dev"."public"."sales-transaction"
WHERE "state or province" = 'California' AND
"Product Sub-Category" ='Office Machines'
```



b. Think of a data cube that would help you answer the questions iv), vii), viii) and ix). Write an SQL query that would create such a cube, and queries on it to answer these three questions, as well as the following two:

1. What is the total profit achieved over all the transactions?
2. How many transactions were there in the Minnesota state?

For creating the data cube (a three dimensional) we need to use 3 columns which are (State or province, Customer segment and product sub-category)

```
create table FirstCube as select "state or province",
```

```
    "product sub-category","customer segment", SUM("profit") as
sumprofit , SUM(Sales)
   from "sales-transaction"
 group by cube("state or province","product sub-
category","customer segment")
```

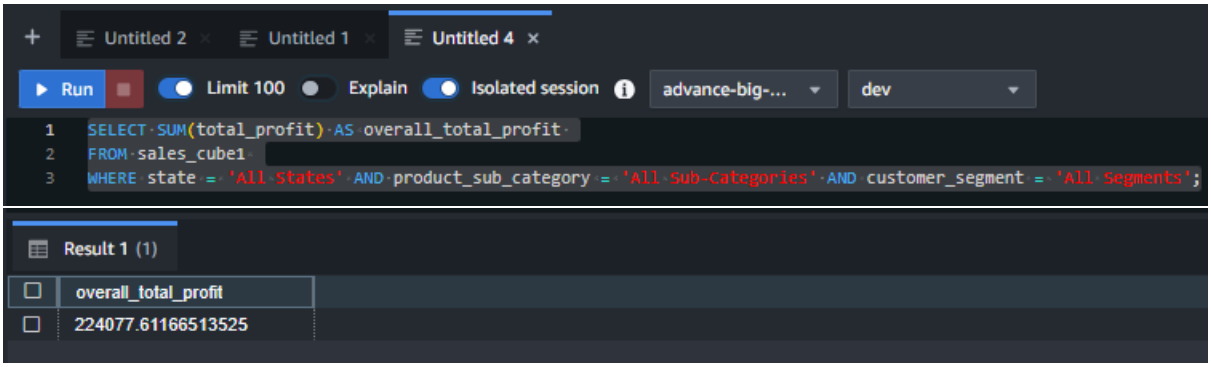
Or

```
CREATE TABLE sales_cube1 AS
SELECT
    COALESCE("state or province", 'All States') AS state,
    COALESCE("product sub-category", 'All Sub-Categories') AS
product_sub_category,
    COALESCE("customer segment", 'All Segments') AS
customer_segment,
    SUM(sales) AS total_sales,
    SUM(profit) AS total_profit,
    COUNT(*) AS transaction_count
FROM
    "sales-transaction"
GROUP BY
    CUBE("state or province", "product sub-category", "customer
segment");
```

Right now, after creating the table we have make a sub-Table for making our performance away better.

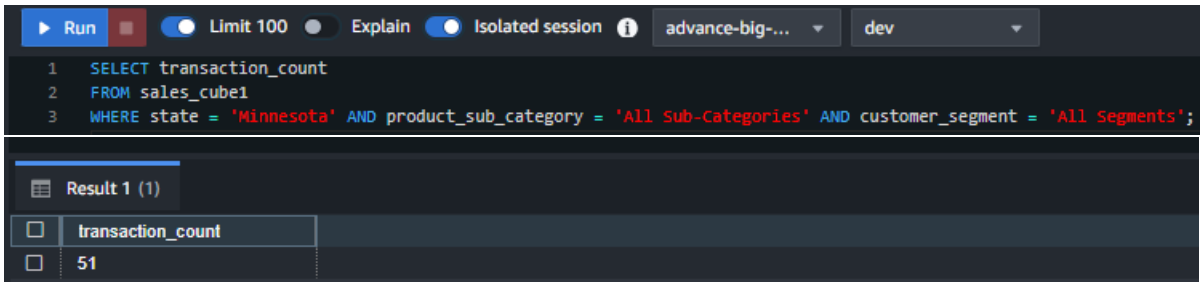
→ Total profit from all transactions is **224077.6**

```
SELECT SUM(total_profit) AS overall_total_profit
FROM sales_cube1
WHERE state = 'All States' AND product_sub_category = 'All Sub-
Categories' AND customer_segment = 'All Segments';
```



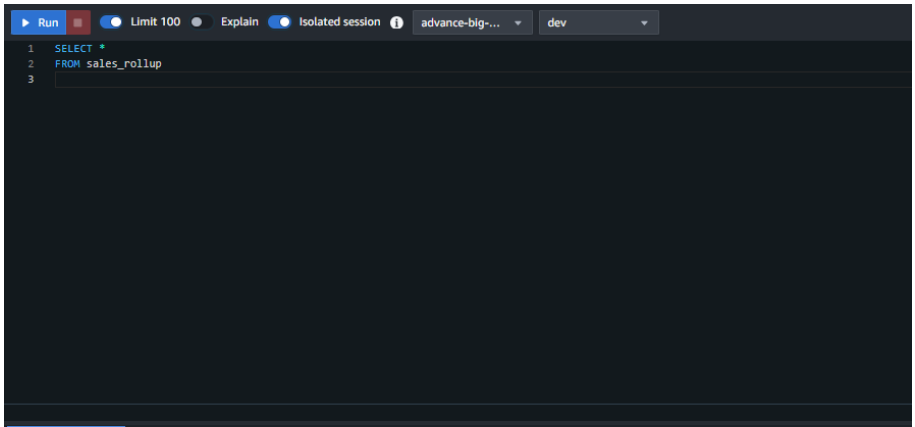
→ Total transactions were there in the Minnesota state is **51**

```sql
SELECT transaction_count
FROM sales_cube1
WHERE state = 'Minnesota' AND product_sub_category = 'All Sub-Categories' AND customer_segment = 'All Segments';
```



c. Create another cube for *roll up* and *drill down* operations that would help you answer the questions ii), iv) and v). Write an SQL query that would create this cube.
   i. How many orders were there in each of the regions?
   ii. Which states ha ve recorded the biggest and the smallest profit?
   iii. Which city in the most profitable state has the least profit?

```sql
CREATE TABLE sales_rollup AS
SELECT
    COALESCE(region, 'All Regions') AS region,
    COALESCE("state or province", 'All States') AS state,
    COALESCE(city, 'All Cities') AS city,
    COUNT(*) AS order_count,
    SUM(profit) AS total_profit
FROM
    "sales-transaction"
GROUP BY
    ROLLUP(region, "state or province", city);
```



region	state	city	order_count	total_profit
West	California	San Gabriel	2	4371.488298416138
East	New York	Syracuse	2	655.3630933761597
West	Washington	Redmond	3	-6965.175298690796
Central	Texas	Round Rock	2	351.5490882396698
West	California	Vacaville	3	2094.7597465515137
South	Louisiana	Terrytown	3	157.53601837158203
East	Massachusetts	Boston	19	4668.203465461731
West	Oregon	Lake Oswego	3	72.9955005645752
East	Pennsylvania	West Mifflin	2	-632.2488698959351
West	Colorado	Fort Collins	3	-4097.763900756836
West	California	San Francisco	4	1619.5809631347656
West	Utah	Lehi	2	-90.02520370483398
Central	Michigan	Detroit	6	1287.2060050964355
West	Utah	Murray	1	-102.51210021972656
Central	Iowa	Newton	6	406.15549755096436
West	Colorado	Fountain	1	35.09000015258789

d. Explain what the benefit is of creating data cubes in parts b) and c) and why are the queries on these data cubes more efficient than queries on the top-level table, as in the part a). What are the drawbacks of creating these cubes?

Creating data Cubes help to improve the efficiencies of querying in the large datasets and some of that benefits are:

1. Minimize the size of the table without effect the data.
2. Minimize the calculating and re-calculation of the usual used query which help to reduce the executing time.
3. Giving a better vision for the table which make it easier to be used and visualize.
4. Support the roll up and down queries.

Disadvantage

1.creating new table will increase the Storage required.

2. less flexibility for the dynamic analysis.

Conclusion

In the assignment we explore how to improve the query by using different solutions such as adding sort key or Global Secondary Index (GSI). Using data cube, it's given more efficient, flexible aggregation of data in different dimensions and that significantly enhanced the performance of the query.

Additionally.

We noticed that using Amazon Redshift was away more expensive than using other features.

References

→ Cesarini, Francesco. *Erlang Programming*. Sebastopol, CA: O'Reilly Media, 2009.

→ Janjic, Vladimir. *Slides for AC51047 - Advanced Big Data Analysis (SEM 1 24/25)*. Lecture, 2024.